

the art of **CODE**

The surprising
power of beauty
in software
development



 MANNING

Sandrine Banas



The Art of Code

The surprising power of beauty in software development

Sandrine Banas

To comment go to [livebook](#).



Manning
Shelter Island

ASPOSE
Your File Format APIs

For more information on this and other Manning titles go to [manning.com](#).

copyright

For online information and ordering of this and other Manning books, please visit [www.manning.com](#). The publisher offers discounts on this book when ordered in quantity. For more information, please contact

Special Sales Department

Manning Publications Co.

20 Baldwin Road

PO Box 761

Shelter Island, NY 11964

Email: orders@manning.com

©2026 by Manning Publications Co. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by means electronic, mechanical, photocopying, or otherwise, without prior written permission of the publisher.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in the book, and Manning Publications was aware of a trademark claim, the designations have been printed in initial caps or all caps.

 Recognizing the importance of preserving what has been written, it is Manning's policy to have the books we publish printed on acid-free paper, and we exert our best efforts to that end. Recognizing also our responsibility to conserve the resources of our planet, Manning books are printed on paper that is at least 15 percent recycled and processed without the use of elemental chlorine.

The authors and publisher have made every effort to ensure that the information in this book was correct at press time. The authors and publisher do not assume and hereby disclaim any liability to any party for any loss, damage, or disruption caused by errors or omissions, whether such errors or omissions result from negligence, accident, or any other cause, or from any usage of the information herein.

Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964

Development editor: Rebecca Johnson
Technical editor: Jean-François Morin
Review editor: Dunja Nikitovic
Production editor: Kathy Rossland
Copy editor: Lana Todorovic-Arndt
Proofreader: Mike Beady
Typesetter: Dennis Dalinnik
Cover designer: Marija Tudor

ISBN: 9781633434929

Printed in the United States of America

dedication

*For my smart, strong, and beautiful daughter
For my family—thank you for your unconditional and loving support*

contents

[***preface***](#)

[***acknowledgments***](#)

[***about this book***](#)

[***about the author***](#)

[***about the cover illustration***](#)

[***1 The aesthetics of code***](#)

[1.1 The patterns of beautiful code](#)

[1.1.1 Storytelling](#)

[1.1.2 Simplicity](#)

[1.1.3 Clarity of intent](#)

[1.1.4 Expressiveness](#)

[1.1.5 Purity](#)

[1.1.6 Sustainability](#)

[1.1.7 Durability](#)

[1.1.8 Creativity](#)

[1.2 A simple example of beautiful code](#)

[1.2.1 Modeling absence](#)

[1.2.2 From absence to beauty](#)

[***2 Narrative code***](#)

[2.1 The plot](#)

[2.1.1 The delivering](#)

[2.1.2 The cleaning](#)

[2.1.3 The defense](#)

[2.1.4 The archiving](#)

[2.1.5 The transformation](#)

[2.2 Code typography](#)

[2.3 High-level storytelling](#)

[2.3.1 Working with legacy code](#)

[2.3.2 Understanding the story](#)

[2.3.3 Simplifying the story](#)

[2.3.4 Levels of narrative: Action, scene, chapter, and table of contents](#)

[2.3.5 Choosing where to break the story](#)

[2.4 The characters](#)

[2.5 The ending](#)

3 The complex art of simplicity

[3.1 Complexity](#)

[3.1.1 Code complexity](#)

[3.1.2 Measuring cognitive complexity](#)

[3.2 Simplicity](#)

[3.2.1 The Prevent lever](#)

[3.2.2 The Organize lever](#)

[3.2.3 The Reduce lever](#)

[3.2.4 The Hide lever](#)

[3.2.5 The Investigate lever](#)

[3.2.6 The Endure lever](#)

4 Expressing clarity of intent through elegant data modeling

[4.1 The pain of working with data in traditional object-oriented languages](#)

[4.1.1 The risk of mutable data](#)

[4.1.2 Understanding the burden of encapsulation](#)

[4.2 Data-oriented approach](#)

[4.2.1 Beauty in record time](#)

[4.2.2 Practical uses for small data carriers](#)

[4.3 Elegant data modeling principles](#)

5 Expressiveness

[5.1 Understanding expressiveness](#)

[5.1.1 Traditional switch in Java](#)

[5.1.2 Java's enhanced switch](#)

[5.1.3 Kotlin if/else](#)

[5.1.4 Kotlin when with a sealed hierarchy](#)

[5.1.5 Haskell pattern matching and direct domain modeling](#)

[5.2 The limits of expressiveness](#)

[6 Embracing purity with functional programming](#)

[6.1 Programming paradigms](#)

[6.1.1 Imperative programming](#)

[6.1.2 Functional programming](#)

[6.2 Functions](#)

[6.2.1 Lambda expressions](#)

[6.2.2 Composition](#)

[6.3 Pipelines](#)

[6.3.1 Understanding functional pipelines](#)

[6.3.2 Exception handling, or not](#)

[6.3.3 Ugly pipelines](#)

[6.3.4 Immutability in pipelines](#)

[6.4 Final match, imperative vs. declarative](#)

[7 Handling failure with grace](#)

[7.1 Categorizing failure](#)

[7.1.1 Business logic errors](#)

[7.1.2 Technical problems](#)

[7.1.3 Programming errors](#)

[7.1.4 Fatal errors](#)

[7.2 Antipatterns in failure handling](#)

[7.2.1 Failing to defend the entry points](#)

[7.2.2 Lazy catch](#)

[7.2.3 Logging gone wrong](#)

[7.2.4 Flawed error handling](#)

8 Sustainability

8.1 The three pillars of sustainability

8.2 The environmental impact of digital technology

8.2.1 The carbon cost of electricity

8.2.2 The hidden lifecycle cost of digital devices

8.3 The craft of green software

8.4 Green design patterns

8.4.1 Frugality pattern

8.4.2 Lean packaging pattern

8.4.3 Lean storage pattern

8.4.4 Lean communication pattern

8.4.5 Efficient execution pattern

8.4.6 Memory efficiency pattern

8.4.7 Carbon-aware pattern

8.5 Tools

8.5.1 Estimating carbon footprint

8.5.2 Tools for developers

9 Durability in software design

9.1 Designing for change

9.1.1 The balance between cohesion and coupling

9.1.2 SOLID

9.1.3 Design patterns

9.2 Resilience

9.3 Testing and the discipline of doubt

9.4 Designing for trust

9.4.1 Designing stable contracts

9.4.2 Evolving without breaking trust

10 Creativity in code and problem-solving

10.1 Creativity in programming

10.1.1 Coding, but not all of it

[10.1.2 Problem-solving](#)

[10.1.3 Creative foresight](#)

[10.2 Creative thinking in coding](#)

[10.2.1 Exercise: Practicing creativity through constraints](#)

[10.2.2 Solutions](#)

[10.3 The creative problem-solving methodology](#)

[10.3.1 Knowledge](#)

[10.3.2 Clarifying](#)

[10.3.3 The creative problem-solving loop](#)

[10.3.4 Implement and reflect](#)

[appendix.](#)

[A.1 Chapter 3](#)

[A.1.1 Calculate cognitive complexity](#)

[A.2 Chapter 4](#)

[A.2.1 Refactoring a class into a record](#)

[A.3 Chapter 5](#)

[A.3.1 From traditional switch to enhanced switch](#)

[A.4 Chapter 6](#)

[A.4.1 Refactor this imperative code using streams](#)

[A.5 Chapter 7](#)

[A.5.1 Spot and refactor antipatterns](#)

[A.6 Chapter 8](#)

[A.6.1 Optimize a customer table](#)

[A.6.2 Greener autocomplete](#)

[A.6.3 Reducing the bandwidth usage of an endpoint](#)

[A.6.4 Optimizing cache memory management](#)

[A.6.5 Making a sensor summary generator greener](#)

[A.7 Chapter 9](#)

[A.7.1 Refactoring for durability](#)

[index](#)

preface

“Wow, that code is ugly,” my former tech lead muttered. He was clearly shocked. I had just walked him through a tricky piece of legacy code written by previous maintainers, helping him navigate it faster. I had previously worked on the same module and spent hours untangling its intricate logic. The code was buried in low-level implementation details, with no clear flow to follow. To be fair, it addressed a complex scenario, but any changes risked introducing side effects. For this reason, developers were reluctant to touch it at all.

After his frustrated remark, I stepped back and let him work through the code to implement a new feature. He worked quietly for hours, fully absorbed. I was stunned when he finally called me back to validate his changes. I expected a patch. Instead, he rewrote the structure entirely. What was once an indecipherable mess had become readable, simple, and elegant in its design. He hadn’t changed the logic, only the way it was told. He rewrote the code as a story, splitting it into small, simple steps, giving each one a meaningful method name and making the overall purpose easy to understand. The difference in readability and maintainability was stark.

I’ve often reflected on that moment. Writing code that was merely bug-free, functional, and maintainable was no longer enough for me. I began to seek a higher standard, one where solving a complex problem with elegance and simplicity brought deep intellectual satisfaction. It was no longer enough to create programs that produced the expected results; the code itself needed to become its own purpose, a finely crafted algorithm, something closer to a work of art that would deliver more than just functionality—lasting quality.

It took me years to articulate this pursuit, and only recently did I dare to use a word that still sparks mixed reactions among developers: *beautiful*. Beauty in code goes beyond indentation or coding conventions. It emerges from deliberate design choices that make code easier to understand and more durable. This book is for developers who want to shift their thinking, approach their work like artists, and, in doing so, improve both the quality of their work and the joy they find in writing code.

acknowledgments

I would like to express my gratitude to the many people who supported and guided me throughout this journey.

First, I thank my development editor, Rebecca Johnson, for her thoughtful advice and constant encouragement. Her guidance played a key role in shaping both the structure and clarity of this book. I am also grateful to my technical editor, Jean-François Morin, for his thorough technical reviews and valuable feedback, which helped ensure the accuracy and rigor of the content. Jean-

François is a senior Java developer and architect at Laval University in Quebec City, Canada.

My sincere thanks go to Michael Stephens, associate publisher, for believing in this book and for his guidance and direction throughout the project. His perspective consistently challenged me to refine and sharpen my definition of beautiful code, helping transform an initial idea into a coherent and grounded work.

I also thank everyone at Manning who contributed behind the scenes to the development, production, and publication of this book.

I am deeply thankful to all the reviewers whose detailed comments and suggestions helped improve the quality, clarity, and correctness of the material: Abe Taha, Allen Ding, Andy Keffalas, Bhagvan Kommadi, Bruce Park, Catalin Roman, Chaitanya Sharma, Claudia Maderthaner, Conor Redmond, Cristina Kiss, Erin Colvin, Greg Wagner, Ilkka Seppälä, Johan Smarius, John Montgomery, Jon Skeet, Jonathan Eastmael Salatan, Juan José Durillo Barrionuevo, Kyle Laverty, Mark Elston, Michael Krimgen, Nancy Al Kalach, Oliver Forral, Orlando Mendez, Oscar Gil, Pax Williams, Rachit Jain, Ramya Boorugula, Robert Delwood, Rohit Purswani, Ruslan Vidzert, Stephen GRIESE, Steven Jenkins, Troi Eisler, Uttam Kumar, and Yev Bronshteyn. Their feedback was invaluable in refining the final manuscript.

I am grateful to my colleagues who shared coding experiences, discussions, and countless merge requests with me over the years. These exchanges continuously shaped my thinking as a developer. I owe special thanks to my colleague Thnouremi Pen, whose shared passion for Java and thoughtful technical discussions consistently challenged my ideas, and who inspired the very first code example that appears in this book.

On a more personal note, I want to thank my family. I am deeply grateful to my daughter for her patience, enthusiasm, and support throughout the long process of writing this book. I also thank my parents, whose unconditional support made everything possible. By buying our first computer at a time when computers were still far from common in households, they unknowingly gave me my first exposure to programming and set me on the path that led to this book. Finally, I thank my big brother, who patiently shared that first computer as we learned BASIC on an Amstrad CPC, making those early programming discoveries even more meaningful.

about this book

The Art of Code is a book for developers who aspire to create software that is both beautiful and enduring. In an era where AI is reshaping the landscape of programming, it refocuses attention on the uniquely human aspects of code: skills, creativity, and beauty. It also explores the broader requirements of

modern enterprise software, including failure handling, resilience, security, durability, and sustainability.

Here, beauty is not an abstract ideal. It's the intentional result of concrete design choices that emphasize simplicity, clarity of intent, purity, expressiveness, and storytelling. Together, these qualities lead to higher-quality software by making code easy to understand, safe to refactor, reliable in production, and durable over time.

Throughout the book, I demonstrate how these principles can be translated into reusable prompts and skills that assist developers in their daily tasks. They can help generate higher-quality code and review existing code. However, they still require human judgment to validate the result, understand the tradeoffs, and take responsibility for the final design.

Regardless of whether you hand-write code or use AI tools, speed must be balanced with human craftsmanship. You bear responsibility for its security and reliability, especially in the most critical and risky parts of a system. Ensuring that your code is both functional and beautiful may require more time upfront, but that investment is repaid many times over by making systems easier to maintain and extend in the long term.

Who should read this book

This book is written primarily for intermediate to senior developers who care about the long-term quality of their code. It is aimed at developers who want to adopt modern coding practices that make systems more durable and sustainable.

Tech leads, senior engineers, and architects will also find this book useful as a mentoring tool. The book provides a shared language and concrete examples to help guide junior developers toward higher-quality code by teaching them how to reason about code and design decisions rather than applying rigid practices or checklists blindly.

While this book is not a beginner's guide to programming, motivated junior developers can still benefit from it. Although some chapters may be challenging, others provide value early in a developer's journey. For example, chapter 7 on failure handling and chapter 10 on creativity and problem-solving can help beginners learn how to handle failure scenarios, approach unfamiliar problems, and work within constraints. Reading this book early can help shape durable habits and a thoughtful mindset.

How this book is organized: A road map

This book is organized around the rosette of beautiful code, a model that brings together the key dimensions of high-quality software. It contains 10 chapters

that progressively explore how these dimensions apply to real-world programming, from local code structure to system-level concerns. Here is a brief overview of each chapter:

This document was truncated here because it was created in the Evaluation Mode.

